

Implementasi Data Warehouse Cuaca 10 Kota di Indonesia Berbasis Open-Meteo API dengan Pipeline ETL Asinkronus dan Analisis OLAP Menggunakan Atoti

Lailil Hidayanti (24031554036), Rahmawati Dian Pratiwi (24031554183), Nurlaila Lutfi Azizah (24051554205)

2024E

Dosen Pengampu: Hasanuddin Al-Habib, M.Si
Program Studi Sains Data, Universitas Negeri Surabaya

Abstrak. Penelitian ini membangun sistem *Data Warehouse* untuk mengolah dan menganalisis data cuaca historis dari 10 kota besar di Indonesia periode 2015–2024 menggunakan *Open-Meteo Historical Weather API*. *Pipeline ETL* diimplementasikan secara *asynchronous* melalui *GitHub Actions*, mencakup ekstraksi 876.720 baris data per jam, transformasi berbasis *star schema* dengan empat tabel dimensi dan satu tabel fakta, serta pemuatan ke PostgreSQL *Supabase* yang dioptimasi dengan *Range Partitioning*, *Materialized View*, dan 15 *indeks*. Analisis multidimensi dilakukan menggunakan Atoti sebagai lapisan *OLAP Cube*, menghasilkan sembilan temuan analitik melalui operasi *drill-down*, *roll-up*, *slice*, dan *thresholding*. Hasil menunjukkan bahwa Surabaya adalah kota terpanas, puncak hujan lebat terjadi pada jam 13:00–14:00, Medan memiliki frekuensi cuaca ekstrem tertinggi, dan penggunaan *Materialized View* mempercepat *query* agregasi sebesar 38 kali dibandingkan *query* langsung ke tabel fakta.

Kata kunci: Data Warehouse, ETL, Open-Meteo API, Star Schema, Supabase, OLAP, Atoti, Cuaca Indonesia

1. Pendahuluan

Indonesia merupakan negara tropis yang memiliki kondisi cuaca yang sangat dinamis. Dalam beberapa tahun terakhir, perubahan cuaca ekstrem terjadi semakin sering dan memberikan dampak pada berbagai sektor, mulai dari lingkungan, pertanian, kehutanan, sumber daya air, infrastruktur, hingga kesehatan masyarakat. Sepanjang tahun 2015–2024 tercatat ribuan bencana yang berkaitan dengan kondisi iklim dan mempengaruhi jutaan penduduk Indonesia. Selain itu, suhu udara di beberapa wilayah juga mencapai tingkat yang sangat tinggi. Kondisi tersebut menunjukkan bahwa pemahaman terhadap pola cuaca di berbagai wilayah Indonesia menjadi hal yang penting untuk mendukung perencanaan dan pengambilan keputusan yang lebih tepat. Namun pemanfaatan data cuaca untuk mendukung pengambilan keputusan tidaklah sederhana. Data cuaca dihasilkan secara berkala dari berbagai lokasi sehingga volumenya terus bertambah dan semakin kompleks untuk dikelola. Proses pengumpulan, integrasi, dan analisis data dalam jumlah besar sering kali kurang efisien apabila dilakukan secara konvensional. Salah satu solusi yang banyak digunakan adalah Data Warehouse. Menurut Inmon, Data Warehouse merupakan kumpulan data yang terintegrasi, berorientasi pada subjek tertentu, bersifat historis, dan dirancang untuk mendukung proses pengambilan keputusan. Berbeda dengan basis data transaksional yang berfokus pada operasional harian, Data Warehouse dirancang untuk kebutuhan analisis data dalam jangka panjang. Salah satu model yang umum digunakan adalah *star schema*, yaitu struktur yang terdiri atas tabel fakta dan beberapa tabel dimensi. Struktur ini memungkinkan proses analisis data dalam jumlah besar dilakukan dengan lebih cepat dan efisien.

Pada penelitian ini, dibangun sebuah workflow Data Warehouse menggunakan data cuaca historis yang diperoleh dari Open-Meteo Historical Weather API. Open-Meteo menyediakan data cuaca historis dalam bentuk *time-series* per jam yang dapat diakses secara terbuka. Data yang digunakan mencakup 10 kota besar di Indonesia, yaitu Jakarta, Surabaya, Bandung, Semarang, Yogyakarta, Medan, Palembang, Makassar, Denpasar, dan Balikpapan, dengan rentang data tahun 2015 hingga 2024. Proses pengolahan data dilakukan melalui tahapan ETL

(Extract, Transform, Load) yang dijalankan secara periodik menggunakan APScheduler dan asyncio. Pada tahap transformasi, dilakukan beberapa proses pengayaan data, seperti pengelompokan intensitas hujan, perhitungan heat index, klasifikasi kondisi cuaca berdasarkan kode WMO, serta pembentukan atribut waktu seperti musim, waktu dalam sehari, dan hari kerja. Selanjutnya, data dimuat ke dalam Supabase yang menggunakan PostgreSQL sebagai basis data utama. Untuk mendukung kebutuhan analisis, diterapkan beberapa fitur optimasi seperti Materialized View, Partitioning, Indexing, serta pemanfaatan ekstensi PostgreSQL. Data yang telah tersimpan kemudian dianalisis menggunakan Atoti sebagai lapisan Data Mart dan OLAP Cube sehingga pengguna dapat melakukan eksplorasi data secara interaktif melalui berbagai visualisasi.

2. Metodologi

2.1 Sumber Data

Penelitian ini menggunakan data cuaca historis yang diperoleh dari [Open-Meteo Historical Weather API](#). Open-Meteo merupakan layanan penyedia data cuaca terbuka yang menyediakan data meteorologi historis maupun prediksi cuaca dalam format time-series dengan resolusi hingga per jam. Data diakses melalui API menggunakan koordinat geografis (*latitude* dan *longitude*) dari setiap lokasi pengamatan. Data yang digunakan mencakup periode 1 Januari 2015 hingga 31 Desember 2024 dengan granularitas per jam (*hourly*). Lokasi pengamatan terdiri atas 10 kota besar di Indonesia, yaitu Jakarta, Surabaya, Bandung, Semarang, Yogyakarta, Medan, Palembang, Makassar, Denpasar, dan Balikpapan. Pemilihan kota dilakukan untuk merepresentasikan wilayah Indonesia bagian barat dan tengah serta variasi karakteristik iklim di berbagai pulau. Variabel yang dikumpulkan meliputi:

Variabel	Jenis DW	Deskripsi
temperature_2m	Measure	Suhu udara aktual pada ketinggian 2 meter dari permukaan tanah
apparent temperature	Measure	Suhu yang dirasakan manusia
relative humidity 2m	Measure	Kelembapan relatif udara
dew point 2m	Measure	Suhu titik embun
precipitation	Measure	Total presipitasi per jam
rain	Measure	Curah hujan cair per jam
cloud cover	Measure	Persentase tutupan awan
wind speed 10m	Measure	Kecepatan angin pada ketinggian 10 meter
wind direction 10m	Measure	Arah angin dalam derajat
wind gusts 10m	Measure	Kecepatan hembusan angin maksimum
shortwave radiation	Measure	Intensitas radiasi matahari
sunshine duration	Measure	Durasi penyinaran matahari
surface pressure	Measure	Tekanan atmosfer permukaan
weather_code	Dimension Attribute	Kode kondisi cuaca standar WMO yang digunakan untuk klasifikasi kategori cuaca
time	Dimension Attribute	Timestamp pengamatan yang digunakan untuk membentuk dimensi waktu dan tanggal
city_name	Dimension Attribute	Lokasi pengamatan yang digunakan untuk membentuk dimensi lokasi

Tabel 1. Struktur Variabel Data

2.2 Proses ETL Extract

Proses ETL Extract merupakan tahap pertama dalam pipeline Data Warehouse yang bertujuan mengambil data cuaca historis dari sumber data eksternal sebelum diproses lebih lanjut (Khan et al., 2024). Pada penelitian ini, proses ekstraksi dilakukan terhadap Open-Meteo Historical Weather API menggunakan bahasa pemrograman Python dengan memanfaatkan library requests dan pandas. Proses ekstraksi diimplementasikan melalui dua fungsi utama.

a. *extract_one_city()*

Bertugas mengirimkan request ke API untuk satu kota dengan parameter berupa latitude, longitude, rentang tanggal pengamatan, daftar variabel cuaca, serta timezone spesifik kota tersebut. Respons API berformat JSON kemudian dikonversi menjadi DataFrame menggunakan library pandas, dilengkapi kolom identitas kota seperti city, province, island, timezone, latitude, longitude, dan elevation.

b. *extract_all_cities()*

Fungsi ini berfungsi mengiterasi seluruh 10 kota secara berurutan, menggabungkan seluruh DataFrame menjadi satu dataset terintegrasi, kemudian menyimpannya ke dalam file CSV sebagai cadangan (*backup*) sebelum memasuki tahap transformasi.

Untuk mendukung kebutuhan pembaruan data secara berkala, proses ekstraksi juga dirancang untuk berjalan secara periodik menggunakan APScheduler. Setiap minggu, fungsi *extract_weekly_update()* secara otomatis menentukan rentang tanggal 7 hari terakhir berdasarkan waktu eksekusi berjalan, kemudian memanggil *extract_all_cities()* untuk mengambil data terbaru dari API. Data yang diperoleh selanjutnya digabungkan dengan data historis yang sudah tersimpan melalui fungsi *append_new_data()*. Mekanisme *drop_duplicates()* berdasarkan kombinasi kolom *time* dan *city* diterapkan untuk memastikan tidak terjadi duplikasi data meskipun terdapat tumpang tindih rentang waktu antar eksekusi. Hasil akhir proses ekstraksi adalah satu dataset terintegrasi yang mencakup 876.720 baris data cuaca hourly dari 10 kota di Indonesia selama periode 2015–2024, siap digunakan pada tahap transformasi.

2.3 Proses ETL Transform

Tahap transformasi merupakan proses pengolahan data mentah hasil ekstraksi menjadi data yang terstruktur, bersih, dan konsisten sebelum dimuat ke dalam Data Warehouse. Menurut Vaisman dan Zimányi (2022), transformasi mencakup tiga aspek utama, yaitu *cleaning* yang menghilangkan kesalahan dan inkonsistensi data ke dalam format yang terstandarisasi, *integration* yang merekonsiliasi data dari berbagai sumber baik pada level skema maupun data, serta *aggregation* yang meringkas data sesuai dengan tingkat granularitas yang dibutuhkan Data Warehouse. Sejalan dengan itu, Inmon (2005) menegaskan bahwa data yang masuk ke dalam Data Warehouse harus dikonversi, diformat ulang, diurutkan kembali, dan diringkaskan sehingga menghasilkan satu representasi fisik yang terintegrasi dan konsisten.

Dalam penelitian ini, proses transformasi diimplementasikan menggunakan bahasa pemrograman Python dengan library utama Pandas dan terdiri atas empat tahapan, yaitu identifikasi anomali data, integrasi dan pembersihan data, *feature engineering*, serta pembentukan tabel dimensi dan tabel fakta sesuai rancangan *star schema*.

a. Identifikasi Anomali Data

Sebelum transformasi dilakukan, data mentah terlebih dahulu diperiksa untuk memastikan kualitasnya. Tahap ini merupakan bagian dari proses *cleaning* yang dalam pandangan Vaisman dan Zimányi (2022) bertujuan mengidentifikasi dan menghilangkan kesalahan serta inkonsistensi agar data dapat digunakan secara andal dalam analisis. Pemeriksaan meliputi kelengkapan data (*missing values*), validitas nilai curah hujan (tidak ada nilai negatif), konsistensi rentang suhu (tidak ada nilai di luar rentang wajar untuk wilayah tropis Indonesia), serta validitas radiasi matahari pada jam-jam yang seharusnya aktif. Meskipun tidak ditemukan anomali signifikan pada dataset, tahap ini tetap penting sebagai bentuk validasi kualitas data sebelum memasuki proses transformasi lanjutan.

b. Integrasi dan Pembersihan Data

Proses integrasi dilakukan untuk menyeragamkan struktur dan format data dari seluruh 10 kota pengamatan. Inmon (2005) menekankan bahwa aspek integrasi adalah karakteristik terpenting dari sebuah Data Warehouse, di mana data dari berbagai sumber yang heterogen harus dilebur menjadi satu representasi yang seragam. Tahap ini diawali

dengan pemecahan kolom time yang semula berbentuk timestamp ISO 8601 (contoh: 2023-01-01T00:00) menjadi dua atribut terpisah, yaitu date dan hour, menggunakan fungsi `str.split()` pada Pandas. Pemisahan tersebut bertujuan memudahkan proses pembentukan dimensi tanggal dan dimensi waktu pada Data Warehouse.

Selanjutnya dilakukan normalisasi zona waktu menggunakan parameter `tz_string` sesuai lokasi masing-masing kota. Kota Jakarta, Surabaya, Bandung, Medan, Semarang, Palembang, dan Yogyakarta menggunakan zona WIB, sedangkan Makassar, Denpasar, dan Balikpapan menggunakan zona WITA. Penyesuaian ini bertujuan memastikan konsistensi representasi waktu antarwilayah sehingga data dapat dianalisis secara terintegrasi. Tahap pembersihan data juga mencakup standarisasi nilai curah hujan dengan memastikan variabel `precipitation` dan `rain` tidak memiliki nilai negatif. Proses ini dilakukan menggunakan fungsi `clip(lower=0)` sehingga seluruh nilai yang digunakan tetap sesuai dengan kondisi fisik yang mungkin terjadi.

c. Feature Engineering

Tahap *feature engineering* bertujuan memperkaya dataset dengan variabel turunan yang mendukung analisis multidimensi. Pendekatan ini sejalan dengan konsep transformasi dalam ETL yang diuraikan oleh Vaisman dan Zimányi (2022), di mana data mentah dari sumber perlu dimodifikasi ke format yang sesuai dengan kebutuhan analisis Data Warehouse. Han, Kamber, dan Pei (2011) dalam *Data Mining: Concepts and Techniques* juga menyatakan bahwa *feature engineering* merupakan proses krusial dalam pengolahan data untuk menghasilkan representasi yang lebih informatif dan bermakna bagi sistem analitik. Variabel-variabel baru yang dibentuk adalah sebagai berikut.

1. `heat_index_diff`, dihitung sebagai selisih antara `apparent_temperature` dan `temperature_2m`, menggambarkan perbedaan antara suhu aktual dengan persepsi panas yang dirasakan manusia.
2. `rain_intensity`, merupakan kategorisasi intensitas hujan berdasarkan nilai `precipitation` ke dalam beberapa kelas seperti "Tidak Hujan", "Hujan Ringan", "Hujan Sedang", dan "Hujan Lebat".
3. `is_rainy_hour`, variabel indikator bertipe boolean yang bernilai True apabila nilai `precipitation` melebihi ambang batas 0,1 mm.
4. `wind_category`, kategorisasi kecepatan angin (`wind_speed_10m`) ke dalam beberapa kelas seperti "Tenang" dan "Semilir" berdasarkan skala Beaufort yang disederhanakan.
5. `direction_label`, konversi arah angin dari satuan derajat menjadi label arah mata angin (N, NE, E, SE, S, SW, W, NW).
6. `is_gust_extreme`, indikator bertipe boolean yang menandai kondisi hembusan angin ekstrem berdasarkan rasio `wind_gusts_10m` terhadap `wind_speed_10m` yang melebihi nilai ambang tertentu.

d. Pembentukan Tabel Dimensi dan Tabel Fakta

Data hasil transformasi selanjutnya diorganisasikan ke dalam struktur *star schema*. Vaisman dan Zimányi (2022) mendefinisikan *star schema* sebagai struktur yang terdiri atas satu tabel fakta sentral yang terhubung ke sejumlah tabel dimensi, di mana tabel fakta memuat *foreign key* ke setiap tabel dimensi beserta nilai-nilai *measures* yang akan dianalisis. Kimball dan Ross (2013) dalam *The Data Warehouse Toolkit* menambahkan bahwa *star schema* dipilih karena kemampuannya mendukung query analitik yang kompleks dengan performa tinggi melalui struktur yang sederhana dan mudah dipahami oleh *end user*. Granularitas merupakan pertimbangan penting dalam desain ini; Inmon (2005) menegaskan bahwa granularitas adalah isu desain tunggal yang paling kritis dalam Data Warehouse, karena semakin rendah level granularitas maka semakin fleksibel query yang dapat dijalankan. Dalam penelitian ini, *star schema* yang dibangun terdiri atas empat

tabel dimensi dan satu tabel fakta, yaitu:

1. Tabel `dim_date` dibentuk dengan mengekstrak seluruh tanggal unik dari kolom `date`, kemudian diperkaya dengan atribut turunan seperti `month`, `year`, `quarter`, `day_of_week`, `is_weekend`, `season`, dan `covid_period`. Atribut `season` dikategorikan berdasarkan pola iklim Indonesia, yaitu DJF (Musim Hujan Puncak), MAM (Transisi 1), JJA (Musim Kemarau), dan SON (Transisi 2).
2. Tabel `dim_time` dibentuk dari 24 nilai jam unik dalam sehari, dilengkapi atribut `time_of_day` dan `is_peak_hour`.
3. Tabel `dim_location` dibentuk dari data geografis 10 kota yang mencakup `city_name`, `province`, `island`, `latitude`, `longitude`, `timezone`, dan `timezone_offset`.
4. Tabel `dim_weather_category` dibentuk dengan memetakan kode cuaca standar WMO ke deskripsi yang mudah dipahami, disertai atribut `severity_level` dan `is_extreme`.

Tabel fakta `fact_weather` dibentuk dengan menggabungkan foreign key dari keempat tabel dimensi ke dalam DataFrame utama melalui operasi `merge`. Tabel ini menyimpan seluruh data pengamatan cuaca per jam per kota, terdiri atas 4 kolom foreign key, 14 kolom measures mentah dari API, dan 6 kolom engineered measures hasil feature engineering.

2.4 Proses ETL Load

Tahap *load* merupakan proses pemindahan data yang telah melalui tahap transformasi ke dalam sistem penyimpanan Data Warehouse secara permanen. Inmon (2005) menjelaskan bahwa data warehouse bersifat *nonvolatile*, artinya data yang dimuat ke dalamnya tidak diperbarui secara langsung seperti pada sistem operasional, melainkan dimuat dalam format *snapshot* yang bersifat statis dan historis. Lebih lanjut, Inmon (2005) menegaskan bahwa setiap unit data dalam Data Warehouse bersifat *time-variant*, yaitu selalu memiliki penanda waktu yang menunjukkan momen akurasi data tersebut. Prinsip ini menjadi dasar perancangan tahap *load* dalam penelitian ini, di mana seluruh data cuaca dimuat beserta atribut waktu yang lengkap dan tidak pernah diubah setelah proses pemuatan selesai.

Dalam penelitian ini, tahap *load* tidak hanya mencakup pemindahan data ke tabel-tabel target, tetapi juga penerapan berbagai mekanisme optimasi database untuk mendukung kebutuhan analisis OLAP. Proses ini diimplementasikan menggunakan PostgreSQL yang dihosting pada platform Supabase sebagai layanan database berbasis *cloud*, dengan koneksi dibangun melalui library SQLAlchemy dan psycopg2 dari Python.

a. Pemuatan Data ke PostgreSQL (Supabase)

Proses pemuatan data dilakukan menggunakan PostgreSQL yang dihosting pada platform Supabase sebagai media penyimpanan utama Data Warehouse. Koneksi ke database dibangun menggunakan library SQLAlchemy dengan psycopg2 sebagai driver PostgreSQL dari Python. Data dimuat ke dalam lima tabel utama yang membentuk struktur star schema, yaitu `dim_date`, `dim_time`, `dim_location`, `dim_weather_category`, dan `fact_weather`. Untuk menjaga integritas referensial, proses pemuatan dilakukan secara berurutan dengan memuat seluruh tabel dimensi terlebih dahulu sebelum tabel fakta, mengingat tabel fakta memiliki foreign key yang merujuk ke keempat tabel dimensi tersebut. Pemuatan data dilakukan menggunakan metode `to_sql()` pada Pandas dengan parameter `if_exists='replace'` untuk memastikan data yang dimuat selalu bersih dari sisa proses sebelumnya.

b. Partitioning Tabel Fakta

Untuk meningkatkan performa *query* berbasis waktu pada data yang bersifat *time-series*, diterapkan teknik *Range Partitioning* pada tabel fakta berdasarkan kolom `date_id`. Vaisman dan Zimányi (2022) mendefinisikan *range partitioning* sebagai strategi yang membagi tuple ke dalam partisi berdasarkan rentang nilai suatu atribut, dan menegaskan bahwa dimensi temporal merupakan kandidat alami untuk strategi ini karena

memungkinkan *query* berbasis rentang waktu diproses secara efisien hanya pada partisi yang relevan. Hal ini sejalan dengan konsep yang dikemukakan Inmon (2005) mengenai pentingnya pengelolaan data historis dalam Data Warehouse, di mana data yang bersifat *time-series* memerlukan strategi penyimpanan yang memudahkan akses berbasis periode waktu. Dalam penelitian ini, terbentuk 120 partisi bulanan yang mencakup periode Januari 2015 hingga Desember 2024, memungkinkan mekanisme *partition pruning* pada PostgreSQL untuk hanya membaca partisi yang relevan tanpa melakukan *scan* pada seluruh data.

c. **Materialized View**

Selain partisi, optimasi juga dilakukan melalui penerapan *Materialized View*. Vaisman dan Zimányi (2022) mendefinisikan *materialized view* sebagai sebuah *view* yang disimpan secara fisik di dalam database, yang meningkatkan performa *query* dengan melakukan pra-komputasi (*precalculating*) terhadap operasi-operasi yang mahal seperti *join* dan agregasi, sehingga *query* yang hanya perlu mengakses *materialized view* dapat dieksekusi lebih cepat.

Pada penelitian ini dibuat dua *Materialized View*, yaitu:

1. *mv_daily_summary*, menyimpan ringkasan data cuaca harian per kota yang mencakup rata-rata, nilai maksimum, dan minimum suhu, rata-rata kelembaban, total presipitasi, rata-rata kecepatan angin, total durasi sinar matahari, serta jumlah jam hujan dalam sehari.
2. *mv_monthly_avg*, menyimpan ringkasan data cuaca bulanan per kota yang mencakup rata-rata dan ekstrem suhu, rata-rata kelembaban, total presipitasi, rata-rata kecepatan angin, total jam hujan, dan total kejadian angin ekstrem.

Keberadaan *Materialized View* memungkinkan proses analisis tren harian maupun bulanan dilakukan dengan waktu eksekusi yang lebih singkat dibandingkan melakukan agregasi langsung pada tabel fakta.

d. **Index dan PostgreSQL Extensions**

Optimasi database juga dilakukan melalui penerapan *index* dan aktivasi ekstensi PostgreSQL. Vaisman dan Zimányi (2022) menjelaskan bahwa mekanisme *indexing* pada Data Warehouse berbeda dari sistem OLTP; dua jenis *index* yang umum digunakan dalam konteks Data Warehouse adalah *bitmap index* yang efektif untuk atribut dengan kardinalitas rendah, dan *join index* yang mematerialisasi *join* relasional antara dua tabel untuk mempercepat akses ke tabel fakta dari dimensi. Dalam penelitian ini, diterapkan tiga kelompok *index*: *index* komposit pada tabel.

1. *index* komposit pada tabel *fact_weather* untuk mempercepat *query join* dan filter berbasis lokasi dan waktu.
2. *partial index* yang hanya mengindeks baris yang memenuhi kondisi tertentu, yaitu *is_rainy_hour* = TRUE dan *is_gust_extreme* = TRUE, sehingga ukuran *index* lebih kecil dan operasi *scan* lebih ringan.
3. *index* pada kedua *Materialized View* untuk mendukung *query* analitik berbasis tahun dan kota. Selain *indexing*, dua ekstensi PostgreSQL juga diaktifkan, yaitu *pg_trgm* untuk mendukung pencarian teks berbasis kemiripan pada kolom kategorikal seperti *city* dan *rain_intensity*, serta *btree_gin* untuk memperluas kemampuan *indexing* multi-kolom menggunakan metode GIN.

2.5 Asynchronous ETL

Untuk memastikan *data warehouse* dapat diperbarui secara berkala tanpa intervensi manual, proses ekstraksi dirancang agar berjalan secara otomatis menggunakan *GitHub Actions* sebagai *scheduler* berbasis *cloud*. Pendekatan ini memungkinkan *pipeline* ETL berjalan secara *asynchronous* tanpa memerlukan server yang aktif secara terus-menerus (ActiveViam, 2024).

Implementasi periodic ETL dilakukan melalui *file* `etl_runner.py` yang dieksekusi oleh *GitHub Actions* setiap minggu secara terjadwal. Untuk melacak progres eksekusi, digunakan tabel `etl_state` pada PostgreSQL *Supabase* yang menyimpan tiga atribut utama, yaitu `run_ke`, `minggu_ke`, dan `last_run`. Setiap kali *pipeline* dijalankan, fungsi `get_weekly_window()` membaca nilai `minggu_ke` dari tabel tersebut untuk menentukan rentang 7 hari berikutnya yang belum diambil. Dengan mekanisme ini, sistem secara otomatis mengetahui periode data yang belum tersimpan tanpa bergantung pada input manual.

Proses deduplikasi dilakukan menggunakan strategi *delete-then-insert*, yaitu menghapus terlebih dahulu seluruh baris pada rentang `date_id` yang akan dimuat sebelum data baru disisipkan. Strategi ini lebih aman dibandingkan pendekatan *upsert* biasa karena memastikan tidak ada duplikasi data meskipun terjadi *re-run* pada periode yang sama. Selain itu, fungsi `ensure_partition()` diterapkan untuk memeriksa dan membuat partisi baru secara otomatis apabila rentang waktu data melewati partisi yang belum tersedia, sehingga struktur partisi tabel fakta tetap terjaga tanpa perlu konfigurasi manual.

2.6 Atoti DataMart

Tahap ini merupakan proses pembangunan *DataMart* berbasis OLAP menggunakan *library* Atoti. Atoti merupakan *library* analitik *Python* yang memungkinkan pembangunan OLAP *cube* secara *in-memory* langsung dari *DataFrame* *pandas*, sehingga eksplorasi data multidimensi dapat dilakukan secara interaktif tanpa memerlukan server OLAP terpisah (ActiveViam, 2024).

Proses diawali dengan memuat ulang seluruh tabel dari *Supabase* PostgreSQL menggunakan `pd.read_sql()` melalui koneksi *SQLAlchemy*. Kelima tabel dimuat secara berurutan, yaitu `dim_location`, `dim_date`, `dim_time`, `dim_weather_category`, dan `fact_weather`. Karena Atoti bekerja dengan satu *DataFrame flat*, kelima tabel tersebut digabungkan menggunakan operasi *left join* bertahap melalui kolom *foreign key* masing-masing, menghasilkan *DataFrame* dengan 876.720 baris dan 47 kolom.

Sebelum data dimuat ke Atoti, dilakukan pemeriksaan dan perbaikan *missing values* yang timbul akibat *unmatched join* pada kolom `weather_cat_id`. Kolom `severity_level` dan `description` yang kosong diisi nilai *default* masing-masing 'Normal' dan 'Tidak Diketahui'. Selain itu, beberapa rekayasa fitur tambahan dilakukan untuk mendukung agregasi dalam *cube* diantaranya yaitu kolom *boolean* `is_extreme` dan `is_rainy_hour` dikonversi menjadi *integer* (0/1), kolom `year_str`, `month_str`, dan `hour_str` dibentuk sebagai representasi *string zero-padded* untuk memastikan urutan label yang benar pada visualisasi, serta `flag_alert_hour` dan `heavy_rain` ditambahkan sebagai indikator kondisi siaga.

Sesi Atoti diinisialisasi menggunakan `tt.Session.start()` yang menjalankan server OLAP lokal secara *in-memory*. *DataFrame* yang telah bersih kemudian dimuat ke dalam Atoti *store* menggunakan `session.read_pandas()` dengan kunci komposit [`date_id`, `time_id`, `location_id`, `weather_cat_id`] yang mencerminkan granularitas data per jam per kota. Dari *store* tersebut, *cube* multidimensi dibuat menggunakan `session.create_cube()` yang secara otomatis mendeteksi seluruh kolom sebagai dimensi potensial. Pendekatan ini sejalan dengan konsep OLAP *cube* dalam *Data Warehouse* yang memungkinkan analisis multidimensi berbasis tabel fakta dan dimensi (Vaisman & Zimányi, 2014). Tahap akhir adalah pendefinisian 10 measures sebagai objek analisis utama menggunakan API `cube.measures`, mencakup ukuran suhu, presipitasi, kelembapan, *heat index*, frekuensi jam hujan, jam cuaca ekstrem, jam badai, dan jam siaga gabungan.

2.6 Atoti Widget

Tahap ini merupakan proses visualisasi dan analisis multidimensi menggunakan OLAP

cube yang telah dibangun pada tahap *Atoti DataMart*. Berbeda dengan pendekatan *widget* interaktif bawaan *Atoti*, visualisasi pada penelitian ini dilakukan secara programatik menggunakan metode *cube.query()* untuk mengekstrak hasil agregasi dari *cube*, yang selanjutnya divisualisasikan menggunakan *library Matplotlib* dan *Seaborn*. Pendekatan ini dipilih karena memberikan fleksibilitas lebih tinggi dalam mengontrol tampilan dan format visualisasi sesuai kebutuhan analisis (Vaisman & Zimányi, 2014).

Analisis dilakukan melalui 9 pertanyaan analitik yang masing-masing mengeksplorasi kombinasi dimensi dan *measures* berbeda. Operasi OLAP yang diterapkan mencakup *drill-down* dari granularitas tahunan ke bulanan dan per jam, *roll-up* dari data *hourly* ke level agregasi harian dan bulanan, *slice* untuk memisahkan data berdasarkan tahun atau kota tertentu, serta *thresholding* untuk mengidentifikasi kondisi ekstrem berdasarkan nilai ambang batas variabel cuaca. Hasil setiap *query* divisualisasikan dalam bentuk *bar chart*, *line chart*, *scatter plot*, dan *heatmap* sesuai karakteristik data yang dianalisis.

3. Hasil dan Analisis

3.1 ETL Extract

Tahap *extract* pada proses ETL dilakukan untuk mengumpulkan data cuaca historis dari Open-Meteo Historical Weather API sesuai kebutuhan analisis yang telah ditentukan. Proses ini bertujuan memperoleh data mentah (*raw data*) dari berbagai lokasi pengamatan dalam format yang seragam sehingga dapat digunakan pada tahap transformasi dan pemuatan data ke dalam Data Warehouse. Data diekstrak menggunakan koordinat geografis (*latitude* dan *longitude*) dari setiap kota sebagai parameter utama permintaan ke API. Proses ekstraksi dilakukan terhadap 10 kota besar di Indonesia, yaitu Jakarta, Surabaya, Bandung, Semarang, Yogyakarta, Medan, Palembang, Makassar, Denpasar, dan Balikpapan, dengan rentang waktu 1 Januari 2015 hingga 31 Desember 2024 pada granularitas per jam (*hourly*). Setiap permintaan menghasilkan respons dalam format JSON yang kemudian dikonversi menjadi DataFrame menggunakan library Pandas. Data dari seluruh kota selanjutnya digabungkan menjadi satu dataset terintegrasi dan disimpan ke dalam file CSV sebagai cadangan (*backup*) sebelum memasuki tahap transformasi.

a. Karakteristik Dataset Hasil Extract

Karakteristik	Nilai
Total Kota	10
Total Baris	876.720
Total Kolom	22
Periode	2015-01-01 s/d 2024-12-31
Granularitas	Per jam (<i>hourly</i>)
Null Values	0
Sumber data	Open-Meteo Historical Weather API

Tabel 2. Karakteristik Dataset Hasil Extract

Berdasarkan Tabel 3.1, sistem berhasil mengambil data cuaca historis dari 10 kota di Indonesia mencakup periode 2 tahun penuh dari tahun 2015 sampai 2024. Pada data tidak ditemukan adanya missing value pada seluruh kolom dataset, sehingga kualitas data awal tergolong baik dan dapat langsung digunakan pada tahap transformasi.

b. Distribusi Data Per Kota

Kota	Jumlah Baris
Surabaya	876.720
Jakarta	876.720
Bandung	876.720
Medan	876.720
Semarang	876.720
Makassar	876.720

Palembang	876.720
Denpasar	876.720
Yogyakarta	876.720
Balikpapan	876.720

Tabel 3. Distribusi Data Per Kota

Berdasarkan tabel 3, menunjukkan bahwa setiap kota memiliki jumlah observasi yang identik, yaitu sebanyak 876.720 record. Keseragaman jumlah data antar kota mengindikasikan bahwa proses ekstraksi berjalan dengan baik dan tidak terdapat kota yang kehilangan data selama periode pengambilan.

c. Struktur Kolom Data Hasil Extract

Kategori	Kolom
Waktu	time
Temperature	temperature 2m, apparent temperature
Kelembapan	relative humidity 2m, dew point 2m
Curah hujan	precipitation, rain
Awan	cloud cover
Angin	wind speed 10m, wind direction 10m, wind gusts 10m
Radiasi	shortwave radiation, sunshine duration
Tekanan udara	surface pressure
Kondisi cuaca	weather code
Lokasi	city, province, island
Geografis	latitude, longitude, elevation

Tabel 4. Struktur Kolom Data Hasil Extract

Dataset hasil extract terdiri dari 22 kolom yang mencakup 14 variabel cuaca yang diperoleh dari Open-Meteo Historical Weather API serta atribut tambahan yang digunakan untuk identifikasi lokasi dan karakteristik geografis. Atribut *time* dan *city* digunakan sebagai dasar pembentukan dimensi waktu dan dimensi lokasi, sedangkan *weather_code* digunakan sebagai atribut untuk mengklasifikasikan kondisi cuaca pada dimensi cuaca, sedangkan kolom numerik lainnya akan menjadi *measures* pada tabel fakta.

d. Hasil Simulasi Periodic ETL

Selain pengambilan data historis, diimplementasikan pula mekanisme periodic extract untuk memastikan data warehouse dapat diperbarui secara berkala. Proses ini menggunakan fungsi *extract_weekly_update()* yang mengambil data cuaca 7 hari terakhir dari API, kemudian digabungkan dengan data historis yang sudah ada menggunakan fungsi *append_new_data()* Untuk membuktikan mekanisme ini berjalan dengan benar, dilakukan simulasi sebanyak tiga kali dengan menggunakan rentang waktu yang berbeda pada setiap iterasi.

Run	Periode	Data Baru	Jumlah Data Sebelum	Jumlah Data Sesudah
1	24–31 Des 2024	1.920	876.720	876.720
2	17–24 Des 2024	1.920	876.720	876.720
3	10–17 Des 2024	1.920	876.720	876.720

Tabel 5. Hasil Simulasi Periodic ETL

Berdasarkan Tabel 5, setiap iterasi berhasil mengambil 1.920 record data cuaca baru dari 10 kota. Namun, jumlah total data setelah proses *append* tidak mengalami perubahan, tetap sebesar 876.720 record. Kondisi ini terjadi karena rentang waktu yang digunakan pada simulasi *periodic* masih berada dalam cakupan dataset historis 2015–2024 yang telah tersimpan sebelumnya. Oleh karena itu, mekanisme *drop_duplicates()* yang diterapkan berdasarkan kombinasi atribut *time* dan *city* mendeteksi seluruh record tersebut sebagai duplikat dan tidak menyimpannya kembali.

3.2 ETL Transform

Tahap transformasi dalam proses ETL dilakukan untuk mengolah data mentah menjadi data yang lebih terstruktur, konsisten, dan digunakan dalam analisis. Pada tahap ini terdapat beberapa proses utama yang meliputi identifikasi anomali data, integrasi serta pembersihan data, dan feature engineering sebagai memperkaya informasi dalam dataset.

a. Identifikasi Anomali Data

Sebelum proses transformasi dilakukan, data mentah terlebih dahulu dianalisis untuk memastikan tidak terdapat anomali yang dapat memengaruhi kualitas hasil akhir. Hasil pemeriksaan menunjukkan bahwa:

1. Kelengkapan data, seluruh 22 kolom dataset tidak memiliki missing values dengan total 876.720 entri lengkap pada setiap kolom.
2. Validitas curah hujan, tidak ditemukan nilai negatif pada variabel *precipitation* maupun *rain*, sehingga data dianggap valid secara fisik.
3. Konsistensi suhu, tidak ditemukan nilai ekstrem di luar rentang wajar ($<10^{\circ}\text{C}$ atau $>45^{\circ}\text{C}$) untuk wilayah Indonesia, yang menunjukkan kestabilan data dari API.
4. Validitas radiasi matahari, variabel *shortwave radiation* tidak menunjukkan nilai nol pada rentang jam 10.00-14.00 yang seharusnya merepresentasikan kondisi siang hari, sehingga tidak terindikasi error pengukuran.

Meskipun tidak ditemukan anomali signifikan, tahap ini tetap penting sebagai bentuk validasi kualitas data sebelum memasuki proses transformasi lanjutan.

b. Integration dan Pembersihan Data

Tahap integrasi dan pembersihan data menghasilkan dataset yang memiliki struktur dan format yang lebih konsisten untuk seluruh kota pengamatan. Proses transformasi yang telah dilakukan memberikan beberapa hasil utama sebagai berikut.

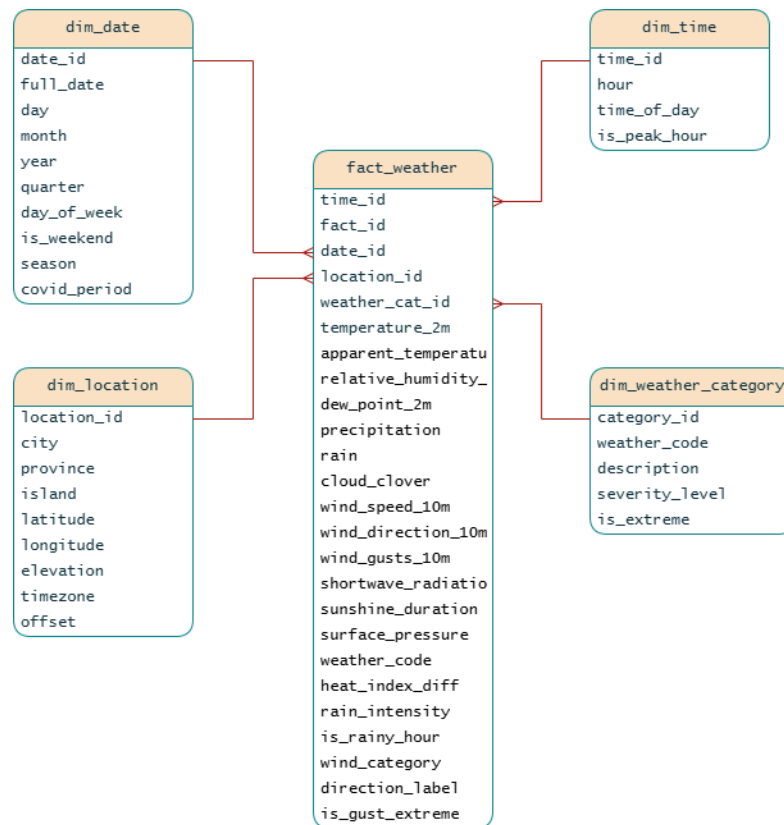
1. Pemecahan kolom time yang sebelumnya tersimpan dalam format ISO 8601 berhasil dipisahkan menjadi dua atribut terpisah, yaitu *date* dan *hour*. Pemisahan ini menghasilkan representasi data temporal yang lebih terstruktur sehingga memudahkan pembentukan dimensi tanggal dan dimensi waktu pada Data Warehouse. .
2. Penyesuaian zona waktu (*time zone normalization*), seluruh data telah berhasil disesuaikan dengan zona waktu masing-masing wilayah pengamatan. Kota-kota yang berada pada zona WIB, seperti Jakarta, Surabaya, Bandung, Medan, Semarang, Palembang, dan Yogyakarta, memiliki representasi waktu yang konsisten dengan wilayah Indonesia bagian barat. Sementara itu, data dari Makassar, Denpasar, dan Balikpapan telah disesuaikan dengan zona WITA.
3. Standarisasi nilai curah hujan, hasil pemeriksaan setelah transformasi menunjukkan bahwa tidak terdapat nilai negatif pada atribut *precipitation* maupun *rain*. Kondisi ini menunjukkan bahwa data curah hujan telah memenuhi validitas nilai yang diperlukan untuk proses analisis selanjutnya.

c. Feature Engineering

Proses *feature engineering* berhasil menghasilkan enam atribut baru yang digunakan untuk memperkaya analisis cuaca dan mendukung kebutuhan OLAP. Atribut yang dihasilkan meliputi *heat_index_diff*, *rain_intensity*, *is_rainy_hour*, *wind_category*, *direction_label*, dan *is_gust_extreme*.

1. *heat_index_diff*, yaitu selisih antara *apparent_temperature* dan *temperature_2m* yang menggambarkan perbedaan antara suhu aktual dan persepsi panas.
2. *rain_intensity*, yaitu kategorisasi intensitas hujan berdasarkan nilai *precipitation*, sehingga data dapat dikelompokkan menjadi kondisi seperti “Tidak Hujan” hingga “Hujan Lebat”.
3. *is_rainy_hour*, yaitu variabel indikator yang menandai apakah suatu jam tergolong

- hujan berdasarkan ambang *precipitation* > 0.1.
4. *wind_category*, yaitu kategorisasi kecepatan angin (*wind_speed*) ke dalam beberapa kelas seperti “Tenang” dan “Semilir”.
 5. *direction_label*, yaitu konversi arah angin dari satuan derajat menjadi label arah mata angin.
 6. *is_gust_extreme*, yaitu indikator kondisi hembusan angin ekstrim berdasarkan rasio *wind_gusts* terhadap *wind_speed* yang melebihi nilai ambang tertentu.
- d. Pembentukan Tabel Dimensi (*Star Schema*) dan Tabel Fakta
- Proses transformasi menghasilkan lima tabel yang membentuk star schema, dengan ringkasan sebagai berikut.



Gambar 1. Dimensi (*Star Schmea*)

1. *dim_date* (3.653 baris) menyimpan atribut turunan setiap tanggal unik dalam dataset, meliputi kolom *date_id*, *full_date*, *day*, *month*, *year*, *quarter*, *day_of_week*, *is_weekend*, *season*, dan *covid_period*. Atribut *covid_period* dikategorikan berdasarkan periode pandemi COVID-19, yaitu Pre-COVID (sebelum 2020), COVID (2020–2022), dan Post-COVID (2023–2024).
2. *dim_time* (24 baris) menyimpan 24 entri setiap jam dalam sehari, dilengkapi *time_of_day* dan *is_peak_hour* untuk menandai jam sibuk.
3. *dim_location* (10 baris) menyimpan informasi geografis 10 kota, meliputi *city_name*, *latitude*, *longitude*, *province*, *island*, *timezone*, dan *timezone_offset*. Adanya perbedaan *timezone* (WIB dan WITA) pada dimensi ini menjadi bukti aspek *integration* dalam Data Warehouse.
4. *dim_weather_category* (15 baris) menyimpan mapping kode cuaca standar WMO ke deskripsi yang mudah dipahami, beserta atribut *severity_level* (Normal, Waspada, Bahaya, Ekstrem) dan *is_extreme*.
5. *fact_weather* (876.720 baris) merupakan tabel fakta utama yang dibentuk dengan

menggabungkan foreign key dari keempat tabel dimensi. Tabel ini terdiri dari 24 kolom yang mencakup 4 kolom foreign key, 14 kolom raw measures dari API, dan 6 kolom engineered measures hasil feature engineering. Setiap baris merepresentasikan kondisi cuaca satu jam di satu kota.

3.3 ETL Load

Setelah proses transformasi selesai, data yang telah bersih dan terstruktur dimuat ke dalam sistem penyimpanan berbasis cloud menggunakan PostgreSQL di Supabase. Tahap load mencakup tidak hanya pemindahan data ke tabel-tabel target, tetapi juga penerapan fitur-fitur lanjutan PostgreSQL seperti partisi, Materialized View, index, dan extensions guna mendukung kebutuhan OLAP secara efisien.

a. Implementasi Proses Load ke PostgreSQL Cloud (Supabase)

Setelah proses transformasi selesai, seluruh data berhasil dimuat ke PostgreSQL yang dihosting pada platform Supabase. Struktur Data Warehouse yang terbentuk terdiri atas empat tabel dimensi dan satu tabel fakta yang saling terhubung melalui relasi *foreign key*. Berdasarkan hasil pemuatan data, seluruh tabel berhasil tersimpan pada PostgreSQL tanpa kendala. Integritas referensial antar tabel juga berhasil dipertahankan sehingga struktur *star schema* dapat digunakan untuk proses analisis dan pembangunan Data Mart pada tahap berikutnya.

b. Partisi Tabel Fakta

Untuk meningkatkan performa query berbasis waktu, dilakukan implementasi *Range Partitioning* pada tabel fakta. Hasil implementasi menghasilkan 120 partisi bulanan yang mencakup periode Januari 2015 hingga Desember 2024. Strategi ini dipilih karena data cuaca bersifat time-series sehingga query analitik umumnya melibatkan filter rentang waktu tertentu, dan partisi berbasis bulan memungkinkan PostgreSQL untuk hanya membaca partisi yang relevan (*partition pruning*) tanpa melakukan scan pada seluruh data. Tabel induk `fact_weather_partitioned` dibuat dengan klausul `PARTITION BY RANGE (date_id)`, kemudian diturunkan menjadi 120 partisi bulanan yang mencakup periode Januari 2015 hingga Desember 2024. Setiap partisi menyimpan data satu bulan untuk seluruh 10 kota, dengan ukuran berkisar antara 1064 kB hingga 1176 kB per partisi. Total data yang berhasil dimuat ke dalam tabel partisi adalah 876.720 baris, konsisten dengan jumlah data pada tabel `fact_weather` asli sehingga tidak ada data yang hilang dalam proses migrasi.

Operation	Cost	Estimated Rows	Actual Time	Actual Rows
SEQ SCAN	(cost 14.3, estimated 430 rows)	430	0.004ms	10 rows
SORT	(cost 34.2, estimated 430 rows)	430	0.58ms	6,697 rows
MERGE JOIN	(cost 612.2, estimated 15,996 rows)	15,996	5.88s	7,440 rows
INDEX SCAN USING FACT_WEATHER_202401_LOCATION_ID_IDX	(cost 320.5, estimated 7,440 rows)	7,440	5.88s	7,440 rows
HASHAGGREGATE	(cost 730.1, estimated 200 rows)	200	0.01ms	10 rows
SORT	(cost 744.3, estimated 200 rows)	200	0.002ms	10 rows

Gambar 2. Hasil Explain Analyze *partition pruning*

Efektivitas partisi dibuktikan melalui `EXPLAIN ANALYZE` pada query filter bulan Januari 2024. Hasil eksekusi menunjukkan bahwa PostgreSQL hanya melakukan *INDEX SCAN* pada partisi `fact_weather_202401` saja (7.440 baris) tanpa menyentuh partisi lainnya, yang membuktikan *partition pruning* berjalan dengan benar. Namun demikian, total waktu eksekusi tercatat 6620ms yang disebabkan oleh operasi *MERGE JOIN* antar partisi hal ini wajar mengingat query melibatkan join dengan tabel dimensi dan agregasi lintas seluruh kota.

c. Materialized View

Implementasi Materialized View berhasil menghasilkan dua tabel agregasi yang digunakan untuk mempercepat proses analisis data cuaca, *Materialized View* yang menyimpan hasil agregasi secara fisik di dalam database sehingga tidak perlu dihitung ulang setiap kali query dijalankan. *mv_daily_summary* menyimpan ringkasan data per hari per kota, mencakup rata-rata, nilai maksimum, dan minimum suhu, rata-rata kelembaban, total presipitasi, rata-rata kecepatan angin, total durasi sinar matahari, serta jumlah jam hujan dalam sehari. View ini menghasilkan 36.530 baris yang merepresentasikan 3.653 hari (2015–2024) $\times$ 10 kota, dan diperuntukkan bagi analisis tren harian serta perbandingan kondisi cuaca antar kota dalam granularitas hari. *mv_monthly_avg* menyimpan ringkasan data per bulan per kota, mencakup rata-rata dan ekstrem suhu, rata-rata kelembaban, total presipitasi, rata-rata kecepatan angin, total jam hujan, dan total kejadian angin ekstrem. View ini menghasilkan 1.200 baris yang merepresentasikan 120 bulan $\times$ 10 kota, dan diperuntukkan bagi analisis tren musiman serta perbandingan antar tahun.

Query Execution Plan			Total time 2425.49ms
>	SORT (cost 13.9, estimated 120 rows)	0.008ms	120 rows
>	INDEX SCAN USING IDX_MV_MONTHLY_YEAR (cost 9.5, estimated 120 rows)	730.27ms	120 rows

Gambar 3. Explain Analyze dengan Mv

Query Execution Plan			Total time 92105.08ms
>	FINALIZE GROUPAGGREGATE (cost 13223.5, estimated 2,400 rows)	1.58s	120 rows
>	GATHER MERGE (cost 13245.5, estimated 2,400 rows)	1.51s	216 rows
>	SORT (cost 13970.5, estimated 2,400 rows)	2.14ms	188 rows
>	PARTIAL HASHAGGREGATE (cost 13834.0, estimated 2,400 rows)	2.18ms	188 rows
>	HASH JOIN (cost 18729.2, estimated 167,551 rows)	87.45s	43,888 rows
>	HASH JOIN (cost 8829.2, estimated 58,624 rows)	87.44s	43,888 rows
>	PARALLEL SEQ SCAN (cost 5917.0, estimated 163,108 rows)	46.50s	87,720 rows
>	HASH (cost 17.4, estimated 163 rows)	0.59ms	365 rows

Gambar 4. Explain Analyze Tanpa Mv

Efektivitas Materialized View dibuktikan melalui perbandingan EXPLAIN ANALYZE antara query langsung ke *fact_weather* (tanpa MV) dan query ke *mv_monthly_avg*. Query tanpa MV membutuhkan waktu 92.105ms (~92 detik) karena harus melakukan *PARALLEL SEQ SCAN* terhadap 87.720 baris dengan dua kali *HASH JOIN* ke tabel dimensi yang masing-masing memakan waktu ~87 detik. Sebaliknya, query menggunakan *mv_monthly_avg* membutuhkan 2.425ms (~2,4 detik) karena PostgreSQL melakukan *INDEX SCAN* menggunakan *idx_mv_monthly_year* terhadap 1.200 baris hasil agregasi. Penggunaan Materialized View memberikan peningkatan performa sebesar ~38x lebih cepat dibandingkan query langsung ke tabel fakta.

d. Index dan Extensions PostgreSQL

Selain partisi dan Materialized View, optimasi database juga berhasil diterapkan melalui aktivasi PostgreSQL Extensions dan pembentukan beberapa indeks pada tabel fakta maupun Materialized View. Untuk meningkatkan performa query OLAP, diterapkan dua jenis optimasi pada level database yaitu extensions dan index. Extensions yang diaktifkan adalah *pg_trgm* dan *btree_gin*. *pg_trgm* mendukung pencarian teks berbasis kemiripan (trigram similarity) yang berguna untuk filter kolom kategorikal seperti *city* dan *rain_intensity*, sedangkan *btree_gin* memungkinkan pembuatan index GIN pada tipe data yang biasanya hanya

mendukung B-tree, sehingga memperluas kemampuan indexing multi-kolom. Index yang diterapkan terbagi menjadi tiga kelompok.

1. index pada fact_weather meliputi index komposit idx_fw_loc_date dan idx_fw_date_loc untuk mempercepat query join dan filter berbasis lokasi dan waktu, serta index pada weather_cat_id untuk filter kategori cuaca.
 2. partial index diterapkan pada kondisi is_rainy_hour = TRUE dan is_gust_extreme = TRUE, berbeda dengan full index, partial index hanya mengindeks baris yang memenuhi kondisi tertentu sehingga ukurannya lebih kecil dan operasi scan lebih ringan.
 3. index pada kedua Materialized View mencakup idx_mv_daily_date, idx_mv_daily_city, idx_mv_daily_year untuk mv_daily_summary, serta idx_mv_monthly_year dan idx_mv_monthly_city untuk mv_monthly_avg. Keseluruhan index yang terbentuk berjumlah 15 index yang tersebar pada tabel fact_weather dan kedua Materialized View.
- e. Performance Benchmark

Pengujian performa dilakukan menggunakan EXPLAIN ANALYZE untuk membuktikan efektivitas penerapan partisi, partial index, dan Materialized View. Tiga skenario pengujian dilakukan dengan hasil sebagai berikut.

Skenario	Metode	Total Time
Partition Pruning	INDEX SCAN 1 partisi	6.620ms
Partial Index (filter hujan)	INDEX ONLY SCAN 3 partisi	297ms
Agregasi bulanan pakai MV	INDEX SCAN mv_monthly_avg	2.425ms
Agregasi bulanan tanpa MV	PARALLEL SEQ SCAN fact_weather	92.105ms

Tabel 6. Hasil Benchmark

1. Skenario 1 Partition Pruning menguji query agregasi suhu dan presipitasi untuk bulan Januari 2024 pada fact_weather_partitioned. Hasil menunjukkan PostgreSQL hanya melakukan *INDEX SCAN* pada partisi fact_weather_202401 tanpa menyentuh 23 partisi lainnya, dengan total waktu eksekusi 6.620ms. Hal ini membuktikan mekanisme *partition pruning* berjalan dengan benar PostgreSQL cukup membaca 7.440 baris dari satu partisi alih-alih seluruh 876.720 baris.
2. Skenario 2 Partial Index menguji query filter jam hujan pada periode Q3 2023 (Juli–September). Hasil menunjukkan PostgreSQL melakukan *INDEX ONLY SCAN* secara langsung pada tiga partisi yang relevan yaitu fact_weather_202307, fact_weather_202308, dan fact_weather_202309 dengan total waktu eksekusi hanya 297ms. Kecepatan ini dicapai karena partial index idx_fwp_rainy hanya mengindeks baris dengan is_rainy_hour = TRUE sehingga jumlah baris yang di-scan jauh lebih kecil, dan PostgreSQL tidak perlu membaca baris yang tidak memenuhi kondisi sama sekali.
3. Skenario 3 Materialized View vs Query Langsung menguji query agregasi bulanan untuk tahun 2023. Query menggunakan mv_monthly_avg selesai dalam 2.425ms melalui *INDEX SCAN* pada idx_mv_monthly_year terhadap 120 baris. Sebaliknya, query langsung ke fact_weather membutuhkan 92.105ms karena harus melakukan *PARALLEL SEQ SCAN* terhadap 87.720 baris disertai dua kali *HASH JOIN* ke tabel dimensi. Penggunaan Materialized View terbukti memberikan peningkatan performa sebesar ~38x lebih cepat dibandingkan query langsung ke tabel fakta.

3.4 ETL Asynchronous

Implementasi periodic ETL berhasil dijalankan sebanyak 5 kali, yang terdiri dari 1 eksekusi terjadwal otomatis (*scheduled*) dan 2 eksekusi manual menggunakan *GitHub Actions* (*workflow_dispatch*), dan 2 eksekusi manual menggunakan *Visual Studio Code* untuk percobaan. *Workflow* dikonfigurasi menggunakan sintaks 'cron: '0 4 * * 5'' yang berarti *pipeline* dijadwalkan berjalan setiap Jumat pukul 04.00 UTC secara otomatis. Kredensial *database* disimpan sebagai *repository secret* (*SUPABASE_URL*) sehingga tidak terekspos dalam kode sumber.

Berdasarkan log eksekusi pada *GitHub Actions*, *run* yang berhasil diselesaikan dengan status sukses berdurasi eksekusi berkisar antara 59 detik hingga 1 menit 6 detik. Eksekusi terjadwal otomatis pertama tercatat pada 5 Juni 2025 pukul 15.01 GMT+7 dengan durasi 1 menit 6 detik, membuktikan bahwa mekanisme *scheduler* berbasis *cron* berjalan sesuai konfigurasi tanpa intervensi manual.

Setiap eksekusi menjalankan urutan proses berikut secara otomatis yaitu, membaca *state* terakhir dari tabel *etl_state* di *Supabase*, menentukan rentang 7 hari berikutnya menggunakan *get_weekly_window()*, mengekstrak data dari *Open-Meteo API* untuk 10 kota, melakukan transformasi dan deduplikasi dengan strategi *delete-then-insert*, memuat data ke tabel fakta yang telah dipartisi, serta memperbarui nilai *etl_state* untuk eksekusi berikutnya. Keseluruhan *pipeline* ini berjalan sepenuhnya *asynchronous* di lingkungan *ubuntu-latest GitHub Actions* tanpa memerlukan server lokal yang aktif.

3.5 Atoti DataMart

Tahap ini merupakan tahap pembangunan *DataMart* berbasis OLAP menggunakan *library Atoti*. Seluruh data hasil ETL yang tersimpan di *Supabase PostgreSQL* dimuat ke dalam sebuah *cube* multidimensi yang memungkinkan analisis cuaca secara interaktif. Proses ini terdiri dari empat tahapan utama yaitu koneksi dan pemuatan data, transformasi dan pembersihan data gabungan, inisialisasi *Atoti session* beserta pembuatan *cube*, serta pendefinisian seluruh *measures*.

a. Koneksi, Pemuatan, dan Penggabungan Data

Koneksi ke *Supabase* dibangun menggunakan *SQLAlchemy* dengan *psycopg2* sebagai *driver PostgreSQL*, mengarah ke *instance Supabase* di *region Asia Tenggara*. Setelah koneksi berhasil, lima tabel dimuat secara berurutan ke dalam *DataFrame* *pandas* menggunakan *pd.read_sql()*. Tabel *dim_date* difilter pada rentang *year BETWEEN 2015 AND 2024*, sedangkan tabel *fact_weather* difilter dengan kondisi *date_id BETWEEN 20150101 AND 20241231* untuk memastikan analisis terbatas pada data historis 10 tahun dan mengecualikan data *pipeline* asinkronus tahun 2025 ke atas. Hasil pemuatan tabel dari *database* adalah sebagai berikut.

Kolom	Jumlah
dim location	10 rows
dim date	3.653 rows
dim time	24 rows
dim weather cat	15 rows
fact weather	876.720 rows

Tabel 7. Tabel Database

Karena *Atoti* bekerja dengan satu *DataFrame* flat, kelima tabel digabungkan menggunakan *left join* bertahap melalui kolom *foreign key* masing-masing. Sebelum *join*, kolom *category_id* pada *dim_weather_category* di-*rename* menjadi *weather_cat_id*, dan tipe data diseragamkan ke *Int64* untuk menghindari *type mismatch* saat *merge*. Hasil akhir *join* menghasilkan *DataFrame* dengan 876.720 baris dan 48 kolom.

b. Pembersihan Data dan Rekayasa Fitur

Setelah join, dilakukan pemeriksaan missing values untuk mengidentifikasi baris yang tidak tertaut ke dimensi tertentu. Hasilnya menunjukkan adanya kolom `severity_level` dan `description` yang kosong akibat unmatched join pada `weather_cat_id`. Kedua kolom tersebut diisi dengan nilai default 'Normal' dan 'Tidak Diketahui'. Setelah proses perbaikan, seluruh missing values berhasil dieliminasi, beberapa kolom rekayasa fitur juga dibuat pada tahap ini untuk mendukung analisis menggunakan atoti widget. Kolom rekayasa fitur tersebut yaitu

1. Kolom boolean `is_extreme` dan `is_rainy_hour` dikonversi dari bool ke integer (0/1) agar dapat langsung dijumlahkan dalam agregasi Atoti.
 2. `year_str`, `month_str`, `hour_str` adalah versi string dari kolom numerik dengan zero-padding ('01', '02', dst.) untuk memastikan urutan yang benar pada visualisasi.
 3. `Alert_hour` dilakukan flag (0/1) bernilai 1 apabila `precipitation > 5 mm` dan `wind_speed_10m > 20 km/h` secara bersamaan, merepresentasikan kondisi siaga gabungan.
 4. `Heavy_rain` dilakukan flag (0/1) bernilai 1 apabila `rain_intensity` sama dengan 'Hujan Lebat'.
 5. Serta kolom era yang mengklasifikasikan data ke dalam tiga periode, yaitu Pre-COVID (2015-2019), COVID (2020-2021), dan Post-COVID (2022-2024).
- c. Inisialisasi Session, Store, dan Cube
- Atoti session dimulai menggunakan perintah `tt.Session.start()` yang menginisialisasi server OLAP lokal, menghasilkan URL akses. DataFrame yang telah bersih kemudian dimuat ke Atoti store menggunakan `session.read_pandas()` dengan kunci komposit `date_id`, `time_id`, `location_id`, dan `weather_cat_id` yang mencerminkan granularitas data per jam per kota. Dari store ini, cube dibuat menggunakan `session.create_cube()` yang secara otomatis mendeteksi semua kolom sebagai dimensi potensial.
- d. Pendefinisian Measures

Tahap terakhir adalah mendefinisikan semua ukuran (*measures*) yang menjadi objek analisis pada *atoti widget*. Seluruh 13 *measures* berhasil didefinisikan dan dikonfirmasi antara lain sebagai berikut.

No.	Nama Measure	Definisi
1	avg temperature	<code>tt.agg.mean(store['temperature 2m'])</code>
2	avg apparent temp	<code>tt.agg.mean(store['apparent temperature'])</code>
3	total precipitation	<code>tt.agg.sum(store['precipitation'])</code>
4	avg humidity	<code>tt.agg.mean(store['relative_humidity 2m'])</code>
5	avg heat index diff	<code>tt.agg.mean(store['heat_index_diff'])</code>
6	avg cloud cover	<code>tt.agg.mean(store['cloud_cover'])</code>
7	avg radiation	<code>tt.agg.mean(store['shortwave_radiation'])</code>
8	avg wind speed	<code>tt.agg.mean(store['wind_speed_10m'])</code>
9	count rainy hours	<code>tt.agg.sum(store['is_rainy_hour'])</code>
10	count heavy rain	<code>tt.agg.sum(store['heavy_rain'])</code>
11	count extreme days	<code>tt.agg.sum(store['is_extreme'])</code>
12	count alert hours	<code>tt.agg.sum(store['alert_hour'])</code>
13	count storm hours	<code>tt.agg.sum(tt.where(store['wind_speed_10m'] > 25, 1, 0))</code>

Tabel 8. Daftar *Measures* yang Didefinisikan pada Atoti Cube

3.6 Atoti Widget

Tahap ini merupakan tahap visualisasi dan analisis multidimensi menggunakan OLAP cube yang telah dibangun pada tahap Atoti DataMart. Tujuh pertanyaan analitik (Q1–Q7) dijawab melalui *query* terhadap *cube* Atoti yang hasilnya divisualisasikan menggunakan

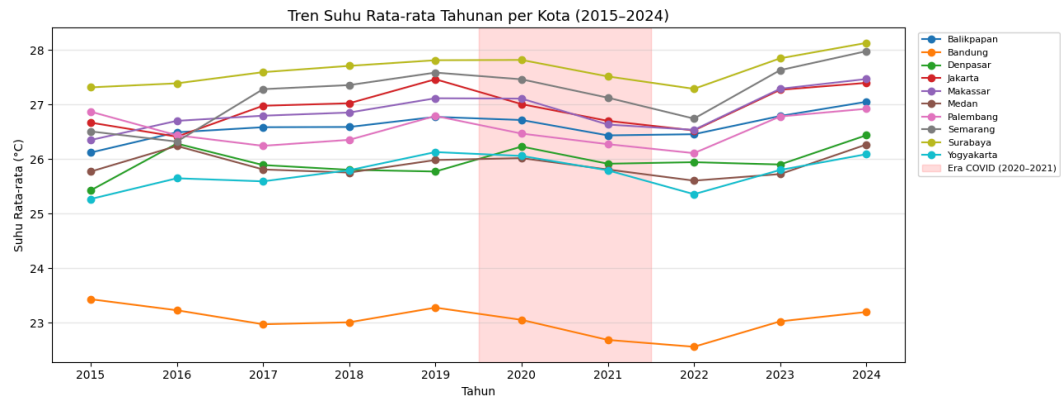
Matplotlib dan *Seaborn*. Setiap visualisasi mengeksplorasi kombinasi dimensi dan measures yang berbeda, mencakup operasi OLAP *roll-up*, *drill-down*, *slice*, dan *thresholding*.

A. Tren Suhu Rata-rata Tahunan 10 Kota (2015–2024)

Analisis ini bertujuan mengidentifikasi tren perubahan suhu rata-rata setiap kota selama 10 tahun (2015–2024). *Query* dilakukan terhadap *measure avg_temperature* dengan dimensi *year_str* dan *city*, menghasilkan 100 baris data (10 kota × 10 tahun). Operasi OLAP yang diterapkan adalah *roll-up* dari granularitas per jam ke level tahunan, dilanjutkan *drill-down* per kota. Visualisasi menggunakan *line chart multi-series* dengan area bayangan yang menyoroti era COVID (2020–2021). Sampel data hasil *query cube* sebagai berikut:

year_str	city	avg_temperature (2015)	avg_temperature (2024)
2015	Balikpapan	26,12	27,05
...	Bandung	23,43	23,20
...	Semarang	26,83	27,98
...	Surabaya	27,37	28,13
2024	Yogyakarta	25,47	26,09

Tabel 9. Sampel Data Suhu Rata-rata Tahunan per Kota



Gambar 5. Line Chart Tren Suhu Rata-rata Tahunan 10 Kota 2015–2024

Gambar 5 menunjukkan bahwa Surabaya secara konsisten menjadi kota terpanas sepanjang 2015–2024 dengan rata-rata mendekati 27,64 °C, diikuti Semarang (27,20 °C) dan Jakarta (26,94 °C). Bandung tetap menjadi kota terdingin di seluruh periode (23,04 °C) karena posisinya di dataran tinggi. Tren kenaikan suhu terlihat hampir di semua kota, dengan kenaikan paling konsisten terlihat pada Surabaya dan Semarang. Bayangan abu-abu pada area 2020–2021 memperlihatkan era COVID yang akan dianalisis lebih lanjut pada Q5.

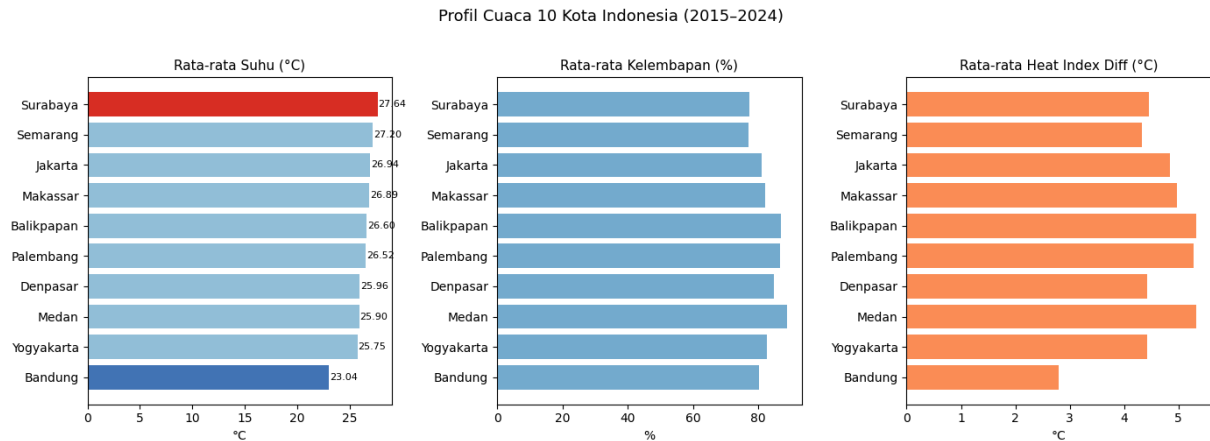
B. Kota Terpanas dan Terdingin: Rata-rata 10 Tahun

Analisis ini melakukan *roll-up* atas seluruh periode 10 tahun untuk meringkas rata-rata suhu, kelembapan, dan heat index per kota. Tiga measures digunakan sekaligus: *avg_temperature*, *avg_humidity*, dan *avg_heat_index_diff*, divisualisasikan dalam tiga subplot horizontal bar chart berdampingan. Data rata-rata 10 tahun per kota hasil *query cube* sebagai berikut:

city	avg_temperature	avg_humidity	avg_heat_index_diff
Bandung	23,04	80,15	2,79
Yogyakarta	25,75	82,61	4,43
Medan	25,90	88,81	5,34

Denpasar	25,96	84,75	4,42
Palembang	26,52	86,56	5,28
Balikpapan	26,60	86,97	5,32
Makassar	26,89	82,15	4,98
Jakarta	26,94	81,01	4,85
Semarang	27,20	76,91	4,33
Surabaya	27,64	77,21	4,46

Tabel 10. Rata-rata Suhu, Kelembapan, dan Heat Index per Kota (2015–2024)



Gambar 6. Horizontal Bar Chart Tiga Dimensi per Kota (Suhu, Kelembapan, Heat Index)

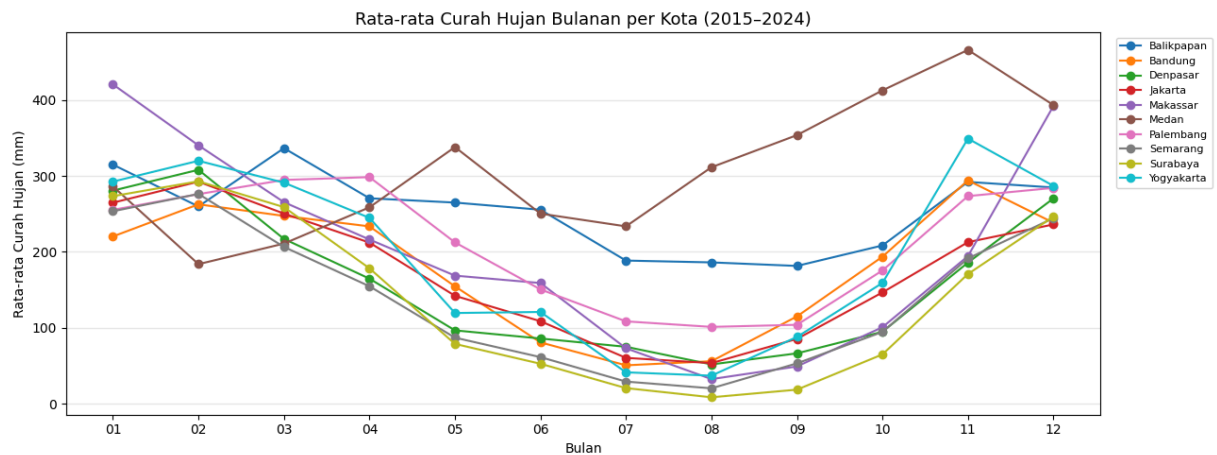
Gambar 6 memperlihatkan bahwa Surabaya merupakan kota terpanas (27,64 °C) dengan heat index diff moderat (4,46 °C), sedangkan Bandung menjadi kota terdingin (23,04 °C) dengan heat index diff terendah (2,79 °C). Menariknya, Medan memiliki heat index diff tertinggi (5,34 °C) sekaligus kelembapan tertinggi (88,81%), mengindikasikan beban panas yang dirasakan tubuh jauh lebih tinggi dari angka termometer. Semarang menunjukkan pola berbeda: suhu kedua tertinggi tetapi kelembapan terendah (76,91%), konsisten dengan karakteristik iklim pesisir utara Jawa yang lebih kering.

C. Pola Musim Hujan Bulanan: Rata-rata 10 Tahun

Analisis pola curah hujan bulanan bertujuan mengidentifikasi kapan puncak musim hujan di masing-masing kota berdasarkan agregasi data 10 tahun. Operasi roll-up dilakukan dari granularitas per jam ke level bulanan dengan menjumlahkan total curah hujan (total_precipitation) per kota per bulan, menghasilkan 120 baris data (10 kota × 12 bulan). Rata-rata bulanan dihitung dengan membagi total 10 tahun dengan 11 (jumlah tahun efektif dalam dataset). Sampel data curah hujan total per bulan hasil query sebagai berikut:

month str	city	total precipitation	avg monthly rain
01	Balikpapan	3.463,1	314,83
01	Bandung	2.422,8	220,25
01	Makassar	4.626,2	420,56
...	...	...	...
12	Yogyakarta	5.888,1	535,28

Tabel 11. Sampel Data Curah Hujan Total per Kota Tiap Bulan



Gambar 7. Line Chart Pola Musim Hujan Bulanan Rata-rata 10 Tahun

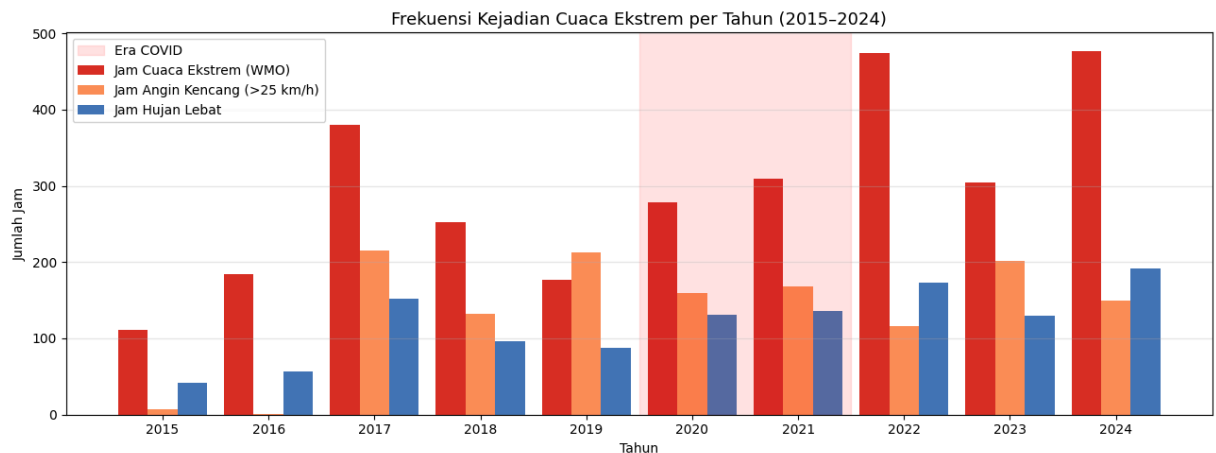
Gambar 7 memperlihatkan dua pola dominan yang berbeda antar kota. Pertama, hampir seluruh kota kecuali Balikpapan dan Medan menunjukkan musim kering yang jelas pada bulan Juni–Agustus dengan curah hujan sangat rendah, lalu melonjak drastis pada November–Maret. Kedua, kota-kota di Kalimantan dan Sumatera (Balikpapan, Medan, Palembang) memiliki distribusi hujan yang lebih merata sepanjang tahun tanpa musim kering yang ekstrem, mencerminkan pengaruh hutan hujan tropis dan posisi geografis yang berbeda. Makassar mencatat curah hujan tertinggi pada bulan Januari (rata-rata 420,56 mm/bulan), konsisten dengan puncak musim barat di kawasan Indonesia Timur.

D. Frekuensi Cuaca Ekstrem per Tahun (10 Kota Gabungan)

Analisis ini menghitung frekuensi tiga indikator cuaca ekstrem secara tahunan untuk seluruh 10 kota: jam cuaca ekstrem berdasarkan klasifikasi WMO (`count_extreme_days`), jam angin kencang di atas 25 km/h (`count_storm_hours`), dan jam hujan lebat (`count_heavy_rain`). Query dilakukan dengan dimensi `year_str` menghasilkan 10 baris data. Visualisasi menggunakan grouped bar chart tiga warna. Data hasil query cube sebagai berikut:

year_str	count extreme days	count storm hours	count heavy rain
2015	111	7	42
2016	184	1	57
2017	380	215	152
2018	252	132	96
2019	177	213	87
2020	278	159	131
2021	309	168	136
2022	474	116	173
2023	304	202	130
2024	477	150	192

Tabel 12. Frekuensi Cuaca Ekstrem per Tahun (10 Kota Gabungan)



Gambar 8. Grouped Bar Chart Frekuensi Cuaca Ekstrem per Tahun 2015–2024

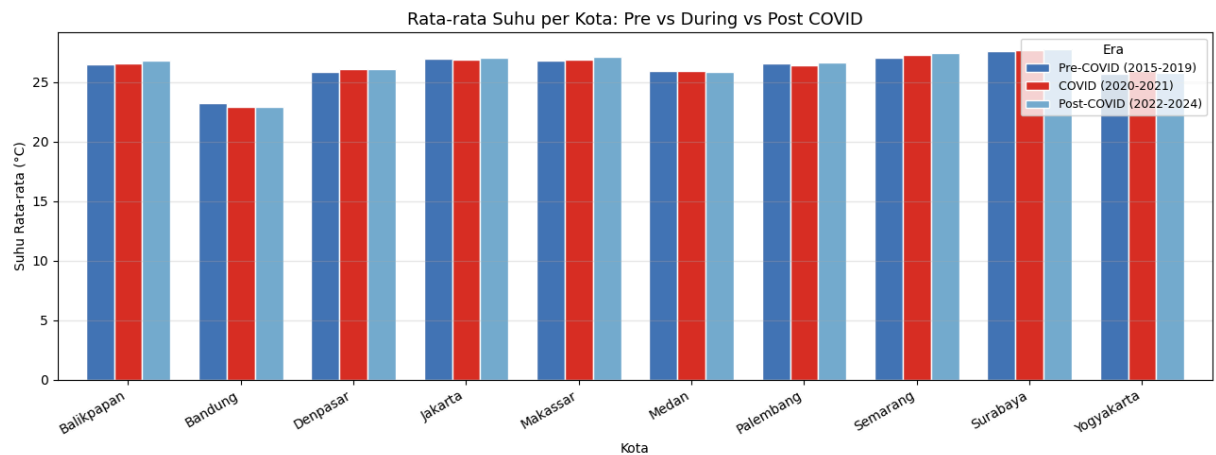
Gambar 8 mengungkapkan tren peningkatan frekuensi cuaca ekstrem yang signifikan. Jam cuaca ekstrem (WMO) melonjak tajam pada 2017 (380 jam) kemudian berfluktuasi, dengan puncak baru di 2022 (474 jam) dan 2024 (477 jam). Jam hujan lebat konsisten meningkat dari 42 pada 2015 menjadi 192 pada 2024, menandakan intensifikasi curah hujan ekstrem dalam satu dekade. Jam angin kencang menunjukkan pola yang lebih tidak beraturan, dengan lonjakan pada 2017 dan 2019 (>200 jam) kemudian mereda. Tren keseluruhan mengindikasikan meningkatnya frekuensi kejadian cuaca ekstrem di Indonesia selama 10 tahun terakhir.

E. [COVID] Perubahan Suhu Rata-rata: Pre vs During vs Post COVID

Analisis ini membandingkan rata-rata suhu per kota di tiga era: Pre-COVID (2015–2019), COVID (2020–2021), dan Post-COVID (2022–2024). Query dilakukan terhadap measure avg_temperature dan avg_humidity dengan dimensi era dan city, menghasilkan 30 baris. Data di-pivot menjadi tabel dengan kota sebagai baris dan era sebagai kolom, divisualisasikan dalam grouped bar chart. Hasil pivot tabel query cube sebagai berikut:

city	Pre-COVID(2015-2019)	COVID(2020-2021)	Post-COVID(2022-2024)
Balikpapan	26,51	26,58	26,77
Bandung	23,18	22,87	22,93
Denpasar	25,84	26,07	26,09
Jakarta	26,91	26,85	27,06
Makassar	26,76	26,87	27,10
Medan	25,91	25,91	25,87
Palembang	26,54	26,37	26,60
Semarang	27,01	27,29	27,45
Surabaya	27,56	27,67	27,76
Yogyakarta	25,69	25,93	25,75

Tabel 13. Rata-rata Suhu per Kota per Era COVID



Gambar 9. Grouped Bar Chart Perbandingan Suhu per Era COVID

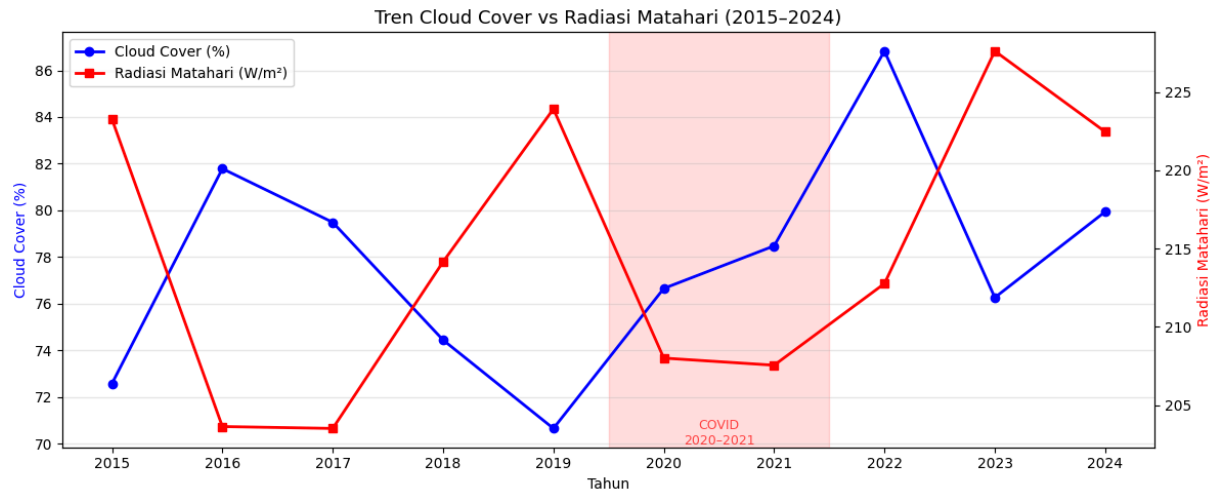
Gambar 9 menunjukkan bahwa hampir semua kota mengalami kenaikan suhu dari Pre-COVID ke Post-COVID, dengan peningkatan paling konsisten pada Semarang (+0,44 °C), Makassar (+0,34 °C), dan Jakarta (+0,16 °C). Menariknya, era COVID (2020–2021) tidak selalu menunjukkan penurunan suhu meskipun aktivitas manusia berkurang. Bandung justru mencatat penurunan kecil selama COVID (22,87 °C) dibandingkan Pre-COVID (23,18 °C), yang bisa disebabkan variabilitas iklim alami. Medan adalah satu-satunya kota yang menunjukkan tren penurunan ringan di era Post-COVID, mengindikasikan dinamika iklim lokal yang berbeda dari kota-kota lainnya.

F. [COVID] Perubahan Cloud Cover dan Radiasi Matahari saat COVID

Analisis ini mengeksplorasi apakah berkurangnya aktivitas industri dan transportasi selama COVID berdampak pada tutupan awan dan intensitas radiasi matahari yang mencapai permukaan bumi. Query dilakukan terhadap `measure avg_cloud_cover` dan `avg_radiation` dengan dimensi `year_str` dan `city`, kemudian diagregasi menjadi rata-rata seluruh kota per tahun. Visualisasi menggunakan line chart dual-axis dengan bayangan era COVID. Data agregasi per tahun hasil query cube sebagai berikut:

year_str	avg_cloud_cover (%)	avg_radiation (W/m ²)
2015	72,59	223,25
2016	81,79	203,62
2017	79,49	203,50
2018	74,45	214,15
2019	70,66	223,92
2020	76,66	208,00
2021	78,48	207,54
2022	86,83	212,76
2023	76,27	227,62
2024	79,94	222,46

Tabel 14. Rata-rata Cloud Cover dan Radiasi Matahari per Tahun



Gambar 10. Line Chart Dual-Axis Cloud Cover vs Radiasi Matahari 2015–2024

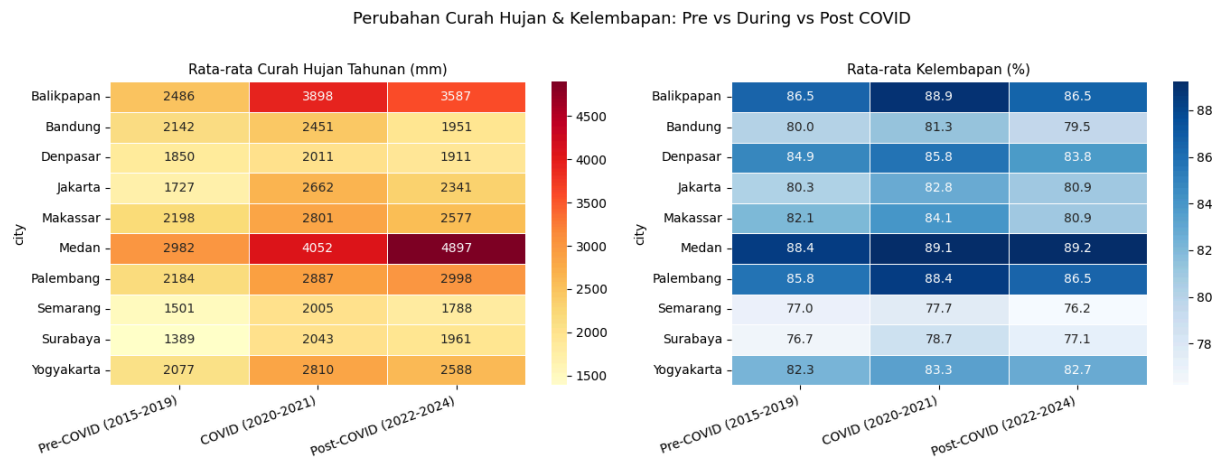
Gambar 10 memperlihatkan pola yang menarik namun tidak langsung mengkonfirmasi hipotesis penurunan polusi. Tutupan awan justru meningkat selama COVID (76,66% pada 2020 dan 78,48% pada 2021) dibandingkan Pre-COVID 2019 (70,66%), sedangkan radiasi matahari turun dari 223,92 W/m² (2019) menjadi 207,54 W/m² (2021). Pola ini lebih konsisten dengan pengaruh La Niña yang meningkatkan curah hujan dan tutupan awan di Asia Tenggara pada periode tersebut daripada efek pengurangan polusi. Pasca-COVID (2022–2024) menunjukkan pemulihan radiasi, dengan 2023 mencatat radiasi tertinggi seluruh periode (227,62 W/m²).

G. [COVID] Perubahan Curah Hujan dan Kelembapan saat COVID per Kota

Analisis ini membandingkan rata-rata curah hujan tahunan dan kelembapan per kota di tiga era COVID. Query dilakukan terhadap measure total_precipitation, avg_humidity, dan count_rainy_hours dengan dimensi era dan city. Total presipitasi dinormalisasi dengan membagi jumlah tahun per era (Pre-COVID: 6 tahun termasuk 2015, COVID: 2 tahun, Post-COVID: 3 tahun) untuk mendapatkan rata-rata tahunan yang sebanding. Visualisasi menggunakan empat subplot heatmap. Sampel data hasil query sebagai berikut:

era	city	avg annual rain (mm)	avg humidity (%)
Pre-COVID (2015-2019)	Balikpapan	2.486,42	86,47
Pre-COVID (2015-2019)	Medan	2.981,72	88,43
COVID (2020-2021)	Balikpapan	3.898,35	88,87
COVID (2020-2021)	Medan	4.051,70	89,13
Post-COVID (2022-2024)	Balikpapan	3.586,67	86,54
Post-COVID (2022-2024)	Medan	4.897,10	89,22

Tabel 15. Rata-rata Curah Hujan Tahunan dan Kelembapan per Kota per Era COVID



Gambar 11. Heatmap Curah Hujan Tahunan dan Kelembapan per Kota per Era

Gambar 11 menunjukkan peningkatan curah hujan yang menonjol di hampir semua kota selama era COVID dibandingkan Pre-COVID. Kota-kota di Kalimantan dan Sumatera (Balikpapan, Medan, Palembang) mencatat lonjakan curah hujan paling signifikan, dengan Medan meningkat dari 2.981,72 mm/tahun (Pre-COVID) menjadi 4.051,70 mm/tahun (COVID) dan bertahan tinggi di 4.897,10 mm/tahun (Post-COVID). Kondisi ini lebih konsisten dengan pengaruh anomali iklim La Niña dan peningkatan aktivitas konvektif selama periode tersebut. Kelembapan juga menunjukkan peningkatan ringan di era COVID, khususnya di Balikpapan (88,87%) dan Medan (89,13%), yang berkorelasi dengan meningkatnya curah hujan dan tutupan awan sebagaimana terlihat pada Q6.

4. KESIMPULAN

Penelitian ini berhasil membangun sistem *Data Warehouse* cuaca berbasis *pipeline* ETL asinkron untuk 10 kota besar di Indonesia. Seluruh tahapan berjalan dengan baik, dengan beberapa kesimpulan utama sebagai berikut.

- Pipeline* ETL berhasil mengekstrak, membersihkan, dan memuat 876.720 baris data cuaca historis 10 tahun (2015–2024) tanpa *missing value* ke dalam struktur *star* schema di *Supabase*, dengan integritas referensial antar tabel yang terjaga.
- Periodic ETL berjalan otomatis setiap minggu melalui *GitHub Actions* tanpa intervensi manual. Strategi deduplikasi *delete-then-insert* memastikan konsistensi data meskipun terjadi *re-run* pada periode yang sama.
- Analisis OLAP selama 10 tahun mengungkap beberapa temuan utama: Surabaya secara konsisten menjadi kota terpanas (rata-rata 27,64 °C) sementara Bandung menjadi yang terdingin (23,04 °C); puncak musim hujan terjadi November–Maret untuk sebagian besar kota, kecuali Balikpapan dan Medan yang memiliki distribusi hujan merata sepanjang tahun; frekuensi cuaca ekstrem menunjukkan tren meningkat dari 111 jam pada 2015 menjadi 477 jam pada 2024; serta Denpasar dan Semarang mendominasi frekuensi angin kencang akibat pengaruh angin muson yang langsung menghantam pesisir kedua kota tersebut.
- Analisis lintas era menunjukkan bahwa hampir seluruh kota mengalami kenaikan suhu rata-rata dari era Pre-COVID ke Post-COVID, dengan peningkatan paling konsisten pada Semarang (+0,44 °C) dan Makassar (+0,34 °C). Peningkatan tutupan awan dan curah hujan selama 2020–2021 lebih konsisten dengan pengaruh anomali iklim La Niña daripada dampak pengurangan aktivitas manusia selama pandemi.
- Sistem yang dibangun memiliki kapasitas pembaruan otomatis dan dapat dijadikan fondasi bagi pengembangan sistem peringatan dini cuaca ekstrem maupun riset meteorologi berbasis data ke depan.

DAFTAR PUSTAKA

- ActiveViam. (2024). Atoti Python SDK documentation. ActiveViam. Retrieved from <https://docs.atoti.io/latest/index.html>.
- Vaisman, A., & Zimányi, E. (2014). Data warehouse systems: Design and implementation. Springer. <https://doi.org/10.1007/978-3-642-54655-6>
- Kurniadi, A., Weller, E., Salmond, J., & Aldrian, E. (2024). Future projections of extreme rainfall events in Indonesia. *International Journal of Climatology*, 44(1), 160–182. <https://doi.org/10.1002/joc.8321>
- Khan, B., Khan, W., Jan, S., & Chughtai, M. I. (2024). An Overview of ETL Techniques, Tools, Processes and Evaluations in Data Warehousing. *Journal on Big Data*, 6(1), 1–20. <https://doi.org/10.32604/jbd.2023.046223>
- Arifani, K. B., Pintarko, D., Sari, A. P., & Agussalim, A. (2025). Real-Time Data Integration and Weather Reporting Automation with Cloud Computing-based Interactive Spatial Dashboard for Extreme Weather Risk Analytics in Indonesia. *Journal of Computer Networks, Architecture and High Performance Computing*, 7(3), 816–824. <https://doi.org/10.47709/cnahpc.v7i3.6298>
- Inmon, W. H. (2005). *Building the Data Warehouse, Fourth Edition*.